

Aeternum-core Smart Contract Audit

Internal Security Audit Report

Contract: AeternumVault.sol

Repository: github.com/Aeternumlabs/aeternum-core

Prepared By: Ndubuisi Ugwuja — Founder, Aeternum Labs

Date: 4th May 2026

Audit Type: Internal Pre-Audit Security Assessment

Status: Complete — External audit pending pre-mainnet

Table of Contents

1. Executive Summary
 2. Project Goals
 3. Contract Architecture & Trust Model
 4. Automated Testing
 5. Static Analysis — Slither
 6. Linting — Solhint
 7. Detailed Findings
 8. Codebase Maturity Evaluation
 9. Known Limitations & Out of Scope
 10. Tool Versions
-

1. Executive Summary

Engagement Overview

AeternumVault is a trustless, non-custodial smart wallet vault with built-in automated ETH recovery powered by Chainlink Automation. Users deposit ETH into a self-sovereign vault, configure a backup address and inactivity period, and if they ever go permanently inactive — lost keys, death, incapacitation — Chainlink Automation transfers their ETH to the backup address automatically. No custodians. No admin backdoors. Just code.

This internal assessment was conducted by the contract author (Ndubuisi Ugwuja — Founder, Aeternum Labs) as a comprehensive pre-audit review prior to Sepolia testnet deployment and eventual professional external audit engagement.

The review employed:

- Static analysis using Slither v0.11.4

- Linting with Solhint
- Manual line-by-line code review
- 109-test Foundry unit test suite achieving 100% line and function coverage
- Foundry fuzz and invariant tests (128,000+ invariant calls)
- Echidna property-based fuzzing across 14 invariant properties with 50,055 randomized call sequences

No High severity vulnerabilities were identified.

Finding Summary

Severity	Count	Status
High	0	—
Medium	2	Acknowledged (mitigated)
Low	1	Resolved
Informational	3	Resolved / Acknowledged

2. Project Goals

This assessment was scoped to provide a thorough internal security review of the AeternumVault contract. The following questions were answered:

- Does the vault correctly enforce user-specific balance isolation, preventing any cross-user fund access?
 - Are all ETH-transferring code paths protected by the Checks-Effects-Interactions (CEI) pattern and ReentrancyGuard?
 - Is the Chainlink Automation integration safe — does checkUpkeep correctly identify due wallets, and does performUpkeep re-validate before acting?
 - Can stale performData from checkUpkeep cause incorrect recoveries or fund loss?
 - Does the swap-and-pop registry removal maintain array integrity under concurrent batch removals?
 - Is the subscription fee accounting correctly isolated from user vault balances?
 - Does the abandonment mechanism after MAX_RECOVERY_ATTEMPTS prevent infinite retry loops that drain Chainlink LINK?
 - Are all access control paths — onlyRegistered, onlyTreasury — enforced correctly?
 - Does the contract correctly reject direct ETH transfers to prevent accidental fund lock?
 - Do all 14 Echidna invariant properties hold across 50,000+ randomized call sequences?
 - Are there any high-severity static analysis findings not previously addressed?
-

3. Contract Architecture & Trust Model

AeternumVault is a single-contract architecture. Each registered wallet has its own isolated vault within the contract — balances are tracked individually per user via a private mapping, never pooled. Chainlink Automation nodes monitor all registered vaults off-chain and execute recovery on-chain only when inactivity conditions are met.

Trust Model

Actor	Can Do	Cannot Do
User	Register, deposit, send, withdrawAll, ping, update config, cancel	Access other users' funds
Treasury	Withdraw accumulated subscription fees	Touch user vault balances
Chainlink Automation	Call performUpkeep when inactivity conditions are met	Alter user configs or redirect funds
Anyone	Call checkUpkeep (read-only, off-chain simulation)	Trigger recovery before period elapses
No one	—	Pause recovery, upgrade the contract, or access user funds

Key Constants

Constant	Value	Description
MIN_INACTIVITY_PERIOD_FREE	365 days	Minimum inactivity period for Free tier
MIN_INACTIVITY_PERIOD_PREMIUM	180 days	Minimum inactivity period for Premium tier
MAX_INACTIVITY_PERIOD	3650 days	Hard ceiling — 10 years
SUBSCRIPTION_DURATION	30 days	Premium subscription window
PREMIUM_MONTHLY_FEE	0.002 ETH	Fee required for Premium registration or renewal
MAX_BATCH_SIZE	50	Maximum wallets processed per performUpkeep call
MAX_RECOVERY_ATTEMPTS	3	Consecutive failures before vault is permanently abandoned

Core Components

User Vault (per address): Each user registers with a backup address and inactivity period. ETH deposited goes into the contract held under their address key. The RecoveryConfig struct tracks: backupAddress, inactivityPeriod, lastActivity, balance, tier, subscriptionExpiry, isActive, failedRecoveryAttempts, and isAbandoned per wallet.

Chainlink Automation: checkUpkeep() is called off-chain every ~12 seconds. It returns a paginated list of wallets due for recovery. performUpkeep() executes on-chain recovery transfers. Both functions support pagination via checkData for scaling to 500,000+ registered wallets across multiple upkeep registrations.

Registry: s_registeredWallets (address[]) tracks all active vaults. s_walletIndexPlusOne provides O(1) position lookup using a 1-indexed mapping, enabling safe swap-and-pop removal during batch operations without corrupting array integrity.

Treasury: s_accumulatedFees tracks subscription revenue separately from user balances. The treasury address can only withdraw this amount — never user vault balances.

Abandonment Mechanism: After MAX_RECOVERY_ATTEMPTS (3) consecutive failed ETH transfers to a backup address, the vault is permanently marked isAbandoned = true and removed from the monitoring registry. The user retains full access to their balance via withdrawAll() or send(). Re-registration of same backup address is blocked.

4. Automated Testing

4.1 Unit Test Coverage (Foundry)

Metric	Result
Line Coverage	100.00%
Statement Coverage	96.89%
Branch Coverage	82.98%
Function Coverage	100.00%
Total Tests	109 passing, 0 failing

Test sections covered: Constructor/Deployment, Registration (Free & Premium), Deposit, Send, WithdrawAll, Ping, Config Updates (backup address, inactivity period), Subscription Renewal, Cancel Recovery, checkUpkeep (pagination, batch size capping, edge cases), performUpkeep (execution, batch recovery, stale data, failure handling), Treasury management, View Helpers, Security (reentrancy attack), Registry Integrity (swap-and-pop), Fuzz Tests, Invariant Tests.

Note: Script files (Deploy.s.sol, HelperConfig.s.sol) and mock contracts (ReentrantAttacker.sol, RejectingReceiver.sol) show 0% coverage as expected — deployment scripts are tested by live execution, not unit tests.

4.2 Fuzz Tests

Four fuzz test functions were run with 256 runs each:

- testFuzz_register_validInactivityPeriod — any period within [FREE_MIN, MAX] accepted
- testFuzz_deposit_amount — any non-zero deposit amount handles correctly

- testFuzz_send_amount — partial sends preserve balance invariants
- testFuzz_isRecoveryDue_timing — recovery only triggers at or after inactivity period

All passed.

4.3 Foundry Invariant Tests

Invariant	Description	Calls	Result
FUND-INV-1	Contract ETH balance >= sum of all user vault balances	128,000	PASS
FUND-INV-2	Accumulated fees never exceed contract ETH balance	128,000	PASS
FUND-INV-3	Every wallet in registry has isActive == true	128,000	PASS

4.4 Echidna Property-Based Fuzzing

Configuration: testMode: property, testLimit: 50,000, seqLen: 100, shrinkLimit: 5,000 **Result:** 50,055 calls executed. All 14 properties passing. Corpus size: 9 sequences saved.

Property	Description	Result
PROP-01	Contract ETH balance covers sum of all user vault balances	PASS
PROP-02	Accumulated fees never exceed contract ETH balance	PASS
PROP-03	Every wallet in registry has isActive == true	PASS
PROP-04	No wallet in registry has isAbandoned == true	PASS
PROP-05	getTotalRegistered() equals actual registry array length	PASS
PROP-06	isActive and isAbandoned are mutually exclusive per wallet	PASS
PROP-07	Every registered wallet has a valid non-zero backup address	PASS
PROP-08	Free tier wallet inactivity period >= MIN_INACTIVITY_PERIOD_FREE	PASS
PROP-09	No wallet with zero balance is flagged as recovery due	PASS
PROP-10	Treasury address is never the zero address	PASS
PROP-11	Free tier wallet subscriptionExpiry is always zero	PASS
PROP-12	Inactivity period never exceeds MAX_INACTIVITY_PERIOD	PASS
PROP-13	failedRecoveryAttempts never exceeds MAX_RECOVERY_ATTEMPTS	PASS
PROP-14	Abandoned wallets are not present in the registry	PASS

5. Static Analysis — Slither

Slither v0.11.4 was run against AeternumVault.sol with --filter-paths "lib/" to exclude third-party dependencies. A total of 16 results were returned across 5 contracts and 101 detectors. No High severity findings.

ID	Detector	Severity	Action
S-01	reentrancy-eth in <code>_executeRecovery</code>	Medium	Acknowledged — silenced with <code>// slither-disable-next-line</code> and full NatSpec rationale
S-02	<code>calls-loop</code> in <code>performUpkeep</code>	Medium	Acknowledged — documented in NatSpec <code>@dev</code> block
S-03	<code>block.timestamp</code> used for inactivity comparisons	Low	Acceptable — 12-second validator manipulation irrelevant for day-scale periods
S-04	costly-loop (pop and delete) in <code>_removeFromRegistry</code>	Informational	Unavoidable — swap-and-pop is already the O(1) optimal pattern
S-05	Inline assembly in <code>checkUpkeep</code>	Informational	Intentional — <code>mstore</code> array trim, documented
S-06	Low-level <code>.call{value}()</code> in 5 functions	Informational	Correct modern pattern — return value checked with <code>if (!success) revert</code>
S-07	OZ library uses <code>^0.8.20</code> pragma	Informational	OZ dependency — not our code. Contract itself pinned to 0.8.24

S-01 — Reentrancy-eth in `_executeRecovery`: Why it is safe

The failure-restore path writes `config.balance`, `config.isActive`, `config.isAbandoned`, and `s_registeredWallets` after the external ETH transfer call. These post-call writes are unavoidable — the contract can only know a transfer failed after making it.

Three independent layers prevent exploitation:

1. `nonReentrant` on `performUpkeep()` blocks all reentrant calls at the outer function level. No reentrant call can reach `_executeRecovery()` during execution.
2. `config.balance` is zeroed BEFORE the external call. Even if reentrancy were somehow possible, there is nothing left to steal — no double-spend is possible.
3. After `MAX_RECOVERY_ATTEMPTS` (3) consecutive failures, the vault is permanently abandoned and removed from the registry. The infinite retry loop is eliminated entirely.

S-02 — Calls Inside Loop: Why it is safe

Batch ETH recovery to distinct backup addresses inherently requires a loop containing external calls. There is no architectural alternative that preserves the automatic recovery guarantee.

Mitigations in place:

- `nonReentrant` on `performUpkeep` prevents reentrant exploitation through any backup address callback
- CEI pattern enforced — all state changes complete before each ETH transfer
- `_executeRecovery` re-validates each wallet individually before acting
- `MAX_BATCH_SIZE` = 50 caps loop iterations within Chainlink's 5,000,000 gas limit

- Failed transfers do not revert the batch — graceful failure handling allows other wallets to proceed

6. Linting — Solhint

Solhint was run with a custom `.solhint.json` configuration tuned to the project's conventions. After remediation, solhint `src/AeternumVault.sol` returns 0 errors and 0 warnings.

Rule	Result	Action
compiler-version	Pass	Pragma pinned to 0.8.24 across all files
var-name-mixedcase	Off	s_ prefix is the established project convention
import-path-check	Off	OZ remapping handled in foundry.toml
function-max-lines	Off	Some complex functions exceed 50 lines intentionally for readability
gas-strict-inequalities	Off	Non-strict inequalities are intentional throughout
use-natspec @param	Fixed	Added @param tags to isRegistered, getRecoveryConfig, isRecoveryDue, getTimeUntilRecovery
use-natspec @notice	Fixed	Added @notice to constructor, _executeRecovery, _removeFromRegistry, receive()

7. Detailed Findings

AV-01 — Reentrancy-eth in _executeRecovery

Severity: Medium **Status:** Acknowledged — Silenced **Category:** Reentrancy **Target:** AeternumVault._executeRecovery()

Description: Slither's reentrancy-eth detector flags `_executeRecovery()` because state variables are written after the external ETH transfer call `backupAddress.call{value: amount}("")`. These writes occur exclusively on the failure-restore path and are unavoidable — the contract can only know a transfer failed after making the call.

Why it is safe:

- `nonReentrant` on `performUpkeep()` blocks all reentrant calls at the outer function level
- `config.balance` is zeroed BEFORE the external call — no double-spend possible
- After `MAX_RECOVERY_ATTEMPTS` (3) failures the vault is permanently abandoned, eliminating the retry loop

Resolution: The Slither detector was silenced with `// slither-disable-next-line reentrancy-eth` with full rationale documented in the NatSpec `@dev` block of `performUpkeep()`. The architectural fix (pull-payment model) was evaluated and rejected because it would require backup addresses to manually call a claim function, breaking the core product promise of automatic recovery without beneficiary action.

AV-02 — External Calls Inside `performUpkeep` Loop

Severity: Medium **Status:** Acknowledged — Documented **Category:** Denial of Service Risk
Target: `AeternumVault.performUpkeep()`

Description: Slither's calls-loop detector flags the ETH transfer to `backupAddress` inside the `performUpkeep` loop. This is an inherent structural requirement.

Mitigations:

- `nonReentrant` prevents reentrant exploitation through any backup address callback
- CEI pattern enforced — all state changes complete before each ETH transfer
- Per-wallet re-validation in `_executeRecovery` prevents stale data exploitation
- `MAX_BATCH_SIZE = 50` caps loop iterations within Chainlink's gas limit
- Failed transfers trigger the failure-restore mechanism, not a batch revert

Resolution: Acknowledged and documented in the `performUpkeep` NatSpec `@dev` block with full rationale.

AV-03 — `send()` Reuses `InvalidBackupAddress` Error for Recipient Validation

Severity: Low **Status:** Resolved **Category:** Semantics / Data Validation **Target:**
`AeternumVault.send()`

Description: The `send(address to, uint256 amount)` function validates that `to != address(0)` but previously reverted with `RecoveryManager__InvalidBackupAddress()`. The recipient in a `send()` is not a backup address — this is semantically incorrect and could confuse frontend error handling.

Resolution: A dedicated `AeternumVault__InvalidRecipientAddress()` custom error was added to `IAeternumVault.sol` and used in `send()`.

AV-04 — Missing NatSpec `@param` and `@notice` Tags

Severity: Informational **Status:** Resolved **Category:** Documentation **Target:** Multiple functions in `AeternumVault.sol`

Description: Solhint identified missing @param tags on isRegistered(), getRecoveryConfig(), isRecoveryDue(), and getTimeUntilRecovery(), and missing @notice tags on the constructor, _executeRecovery(), _removeFromRegistry(), and receive().

Resolution: All missing NatSpec tags were added during the audit remediation pass. The contract now has complete NatSpec coverage across all public, external, and internal functions.

AV-05 — OpenZeppelin Dependency Uses ^0.8.20 Pragma

Severity: Informational **Status:** Acknowledged **Category:** Configuration **Target:** lib/openzeppelin-contracts/contracts/utils/ReentrancyGuard.sol

Description: OpenZeppelin's ReentrancyGuard.sol and StorageSlot.sol use pragma solidity ^0.8.20 which Slither flags as containing known severe issues: VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, and MissingSideEffectsOnSelectorAccess.

Why it is acceptable: AeternumVault.sol is pinned to pragma solidity 0.8.24 which resolves all three issues. The OZ files are third-party dependencies compiled under our pinned compiler version. They cannot be modified.

8. Codebase Maturity Evaluation

Category	Rating	Notes
Arithmetic	Satisfactory	Solidity ^0.8.24 native overflow protection. unchecked{} used only in loops with explicit bounds.
Auditing & Logging	Satisfactory	12 events covering all state-changing operations: RecoveryRegistered, Deposited, Sent, Withdrawn, RecoveryExecuted, RecoveryFailed, RecoveryAbandoned, RecoveryCancelled, SubscriptionRenewed, TreasuryUpdated, SubscriptionFeesWithdrawn, ActivityPinged.
Access Controls	Strong	onlyRegistered and onlyTreasury modifiers correctly gate all functions. No admin backdoors. Treasury strictly isolated to subscription fees.
Complexity Management	Satisfactory	Clean separation: user functions, Chainlink automation functions, treasury functions, internal helpers. Interface fully separates types from implementation.
Decentralization	Strong	No upgradeability. No admin pause. No owner override. Trustless by design — only the user and pre-authorized backup can ever receive funds.

Category	Rating	Notes
Documentation	Satisfactory	Comprehensive NatSpec. Full @param/@notice/@dev coverage. Trust model, CEI pattern, and security considerations documented inline.
Low-Level Manipulation	Satisfactory	Single inline assembly block in checkUpkeep for array trimming (mstore). Justified for gas efficiency. Documented and acknowledged.
Testing & Verification	Strong	109 unit tests, 4 fuzz tests, 3 Foundry invariants (128k+ calls), 14 Echidna properties (50k+ calls), Slither, Solhint. 100% line and function coverage.
Reentrancy	Satisfactory	ReentrancyGuard on all ETH-transferring functions. CEI enforced throughout. One acknowledged post-call write on failure-restore path — mitigated by three independent layers.
Transaction Ordering	Satisfactory	performUpkeep re-validates each wallet before acting. Stale checkUpkeep data silently skipped. No flash loan or front-running attack surface identified.

9. Known Limitations & Out of Scope

The following items are known limitations of this internal assessment and should be addressed by an external professional audit prior to mainnet deployment:

- ERC20 token support is not yet implemented. The current contract handles native ETH only. Future versions will add ERC20 deposits with per-token balance tracking.
- The watcher service backend (Node.js/TypeScript) and frontend (Next.js) are outside the scope of this contract audit and require separate security review.
- This assessment was conducted by the contract author. An independent external audit is required before mainnet deployment.
- Chainlink Automation upkeep registration and LINK funding have not yet been validated on Sepolia testnet.
- Mainnet gas cost estimates have not been validated under production fork conditions.
- The Echidna fuzzing corpus covers 50,055 calls across 14 properties. Longer runs with higher testLimit values are recommended before external audit engagement.
- The contract does not implement ERC-4337 account abstraction. This is a deliberate design decision.

10. Tool Versions

Tool	Version	Purpose
Foundry forge	Latest stable (May 2026)	Unit, fuzz, and invariant testing

Tool	Version	Purpose
Slither	0.11.4	Static analysis
Solhint	Latest stable	Solidity linting
Echidna	Latest (Docker — ghcr.io/crytic/echidna)	Property-based fuzzing
Solc	0.8.24	Solidity compiler
OpenZeppelin Contracts	v5.x	ReentrancyGuard dependency

This internal audit report was prepared by Ndubuisi Ugwuja (Aeternum Labs) as a pre-audit security assessment. It does not constitute a professional third-party audit. An independent external audit is required before mainnet deployment. © 2026 Aeternum Labs. All rights reserved.